

WEEK 3

DAA ASSIGNMENT

Kruskal's Algorithm

NAME – Parika Sharma

SAPID- 590024391

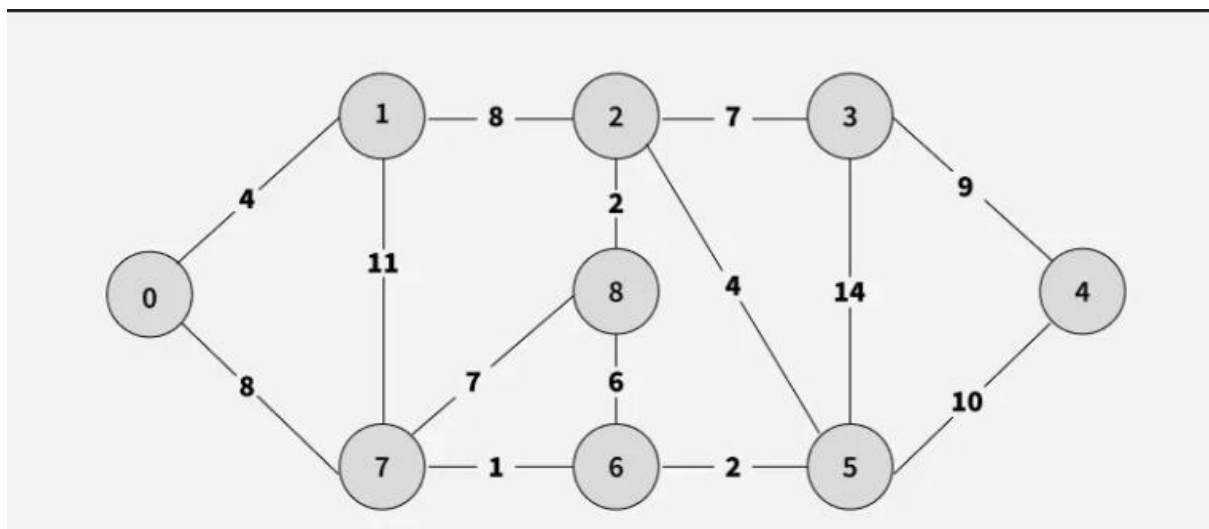
Kruskal's Algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.

The algorithm works by arranging all edges in increasing order of their weights. It then selects the smallest-weight edge one by one, provided that adding the edge does not create a cycle.

The process continues until the spanning tree contains $V - 1$ edges.

Basic steps

1. Sort all edges according to their weight in ascending order.
2. Select the edge with the smallest weight.
3. Check whether adding the edge creates a cycle.
4. If it does not create a cycle, add it to the spanning tree.
5. Repeat until $V - 1$ edges have been selected.



Spanning Tree

A **spanning tree** of a connected, undirected graph is a subgraph that contains **all the vertices** of the original graph and has **no cycles**.

For a graph with **V vertices**, a spanning tree always contains exactly **$V - 1$ edges**.

Important properties

- It contains all vertices of the graph.
- It contains no cycles.
- There is exactly one path between any two vertices.
- A connected graph can have more than one spanning tree.

C CODE OF KRUSKAL'S ALGORITHM

```
#include <stdio.h>

#define MAX 100

struct Edge {
    int u, v, weight;
};

int parent[MAX];

int find(int i)
{
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int u, int v)
{

```

```

int rootU = find(u);
int rootV = find(v);

parent[rootU] = rootV;
}

void sortEdges(struct Edge edges[], int n)
{
    struct Edge temp;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (edges[j].weight > edges[j + 1].weight)
            {
                temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

int main()
{
    int V, E;

    struct Edge edges[MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");

```

```

scanf("%d", &E);

printf("Enter edges (source destination weight):\n");

for (int i = 0; i < E; i++)
{
    scanf("%d %d %d",
        &edges[i].u,
        &edges[i].v,
        &edges[i].weight);
}

for (int i = 0; i < V; i++)
    parent[i] = i;

sortEdges(edges, E);

int count = 0;
int totalWeight = 0;
printf("\nEdges in Minimum Spanning Tree:\n");

for (int i = 0; i < E && count < V - 1; i++)
{
    int u = edges[i].u;
    int v = edges[i].v;

    if (find(u) != find(v))
    {
        printf("%d -- %d = %d\n",
            u, v, edges[i].weight);
    }
}

```

```
        totalWeight += edges[i].weight;

        unionSet(u, v);

        count++;
    }
}

printf("Total weight of MST = %d\n", totalWeight);

return 0;
}
```

```
Enter number of vertices: 5
Enter number of edges: 6
Enter edges (source destination weight):
0 1 1
1 3 2
0 3 3
0 2 4
2 3 5
3 4 5

Edges in Minimum Spanning Tree:
0 -- 1 = 1
1 -- 3 = 2
0 -- 2 = 4
3 -- 4 = 5
Total weight of MST = 12
```